

Интерфейсы в Go

Матвей Ефимов
Эксперт в Go, Python, SQL

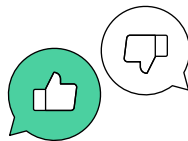


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Матвей Ефимов

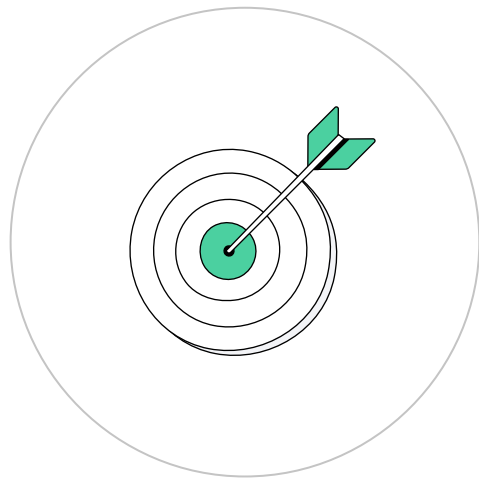
О спикере:

- Преподаватель с более чем 5-летним опытом в IT-образовании
- Разработчик учебных программ и эксперт в Go, Python, SQL
- Фриланс-программист и наставник студентов от школ до вузов



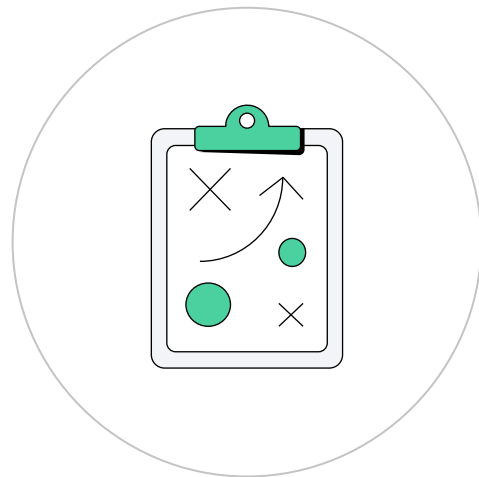
Цели занятия

- Разберёмся, что такое интерфейсы в Go, и чем они отличаются от интерфейсов в других языках
- Поймём, для чего нужны интерфейсы и как они помогают реализовывать полиморфизм
- Научимся объявлять и реализовывать интерфейсы
- Познакомимся с пустым интерфейсом
- Разберём стандартный интерфейс error
- Изучим частые ошибки и плохие практики, связанные с интерфейсами



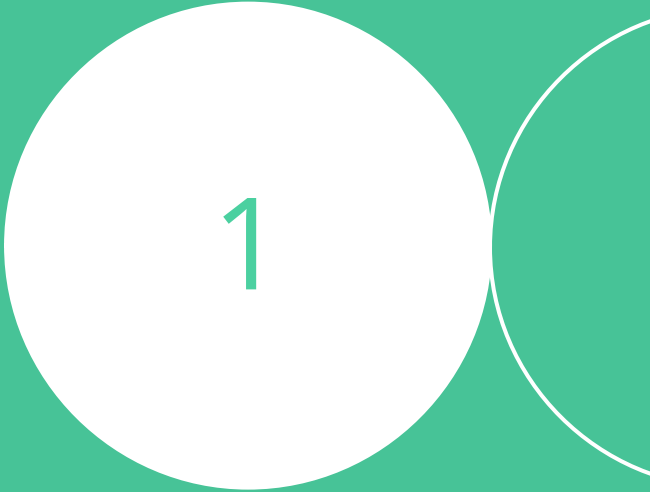
План занятия

- 1 [Основы интерфейсов](#)
- 2 [Объявление и реализация интерфейса](#)
- 3 [Полиморфизм в языке Go](#)
- 4 [Пустой интерфейс и интерфейс error](#)
- 5 [Частые ошибки и лучшие практики](#)



*Нажмите на нужный раздел для перехода

Основы интерфейсов



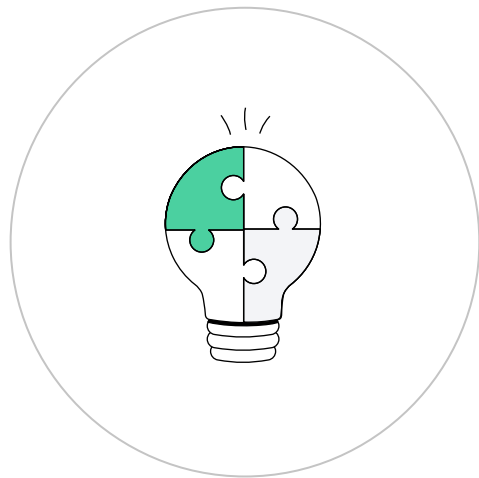
1

Интерфейс в Go

— это набор методов. Он описывает поведение, но не содержит реализацию этого поведения

Неявная реализация интерфейса

В Go нет необходимости явно указывать, что структура реализует какой-то интерфейс. Если у структуры есть нужные методы — она автоматически считается реализацией интерфейса. Это называется **неявной реализацией интерфейса**



Особенности интерфейсов в Go

- 1 Неявная реализация
- 2 Интерфейсы не содержат состояния
- 3 Интерфейс может содержать любой набор методов
- 4 Интерфейсы можно вкладывать друг в друга
- 5 Интерфейсы — это ссылочный тип

Пример простого интерфейса

```
package main

import "fmt"

type Speaker interface { // Объявляем интерфейс
    Speak()
}

type Dog struct{} // Объявляем структуру

func (d Dog) Speak() { // Реализуем метод Speak для Dog
    fmt.Println("Гав-гав!")
}

func main() {
    var s Speaker // переменная интерфейсного типа
    s = Dog{}     // присваиваем значение структуры

    s.Speak()     // вызов метода через интерфейс
}
```

Разница между структурой и интерфейсом

1 Интерфейс — это абстракция, описание поведения

```
type Mover interface {  
    Move()  
}
```

→ Структура — это конкретный тип с данными

```
type Person struct {  
    Name string  
    Age  int  
}
```

Разница между структурой и интерфейсом

2 Интерфейс может ссылаться на разные типы

```
var s Speaker
s = Dog{}
s.Speak()    // "Гав-гав!"

s = Person{Name: "Иван"}
s.Speak()    // "Привет, меня зовут Иван!"
```

→ Структура ничего не знает про интерфейсы, которые реализует

Неявная реализация

Пример:

```
type Printer interface {  
    Print()  
}  
  
func SendToPrint(p Printer) {  
    p.Print()  
}
```

Такой подход делает код универсальным, гибким и удобным для тестирования

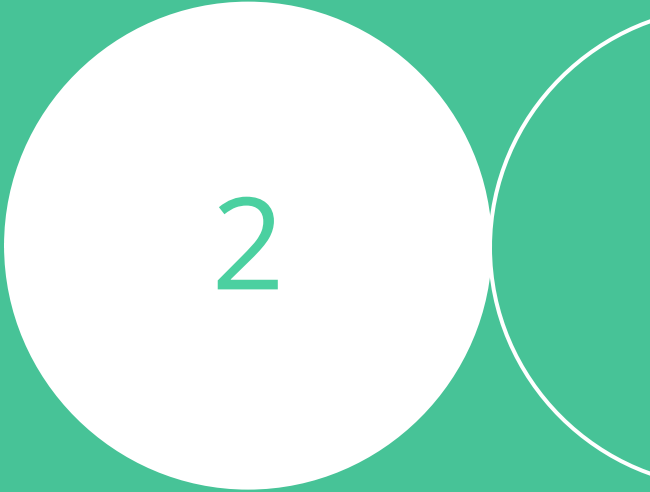
Подведём итоги

- Интерфейс — это набор методов, которые должен реализовать тип
- Структура — это набор данных. Структуры реализуют интерфейсы, если у них есть нужные методы
- В Go реализация интерфейса происходит неявно — без деклараций
- Интерфейсы позволяют писать обобщённый код, который не зависит от конкретных типов



Ваши вопросы?

Объявление и реализация интерфейса



2

Объявление интерфейса

В языке Go интерфейсы объявляются при помощи ключевых слов **type** и **interface**.

Базовый синтаксис:

```
type ИмяИнтерфейса interface {  
    Метод1(параметры) возвращаемые_значения  
    Метод2(параметры) возвращаемые_значения  
}
```

Объявление интерфейса

Пример:

```
type Speaker interface {  
    Speak()  
}
```

Обратите внимание: у интерфейсов нет тела метода. Мы не описываем, как метод работает

Объявление интерфейса

Интерфейс может содержать несколько методов:

```
type Animal interface {  
    Speak()  
    Walk()  
}
```

В Go для соответствия интерфейсу метод должен совпадать **и по имени**, и по **сигнатуре**, то есть по типам параметров и возвращаемых значений.

Если метод написан без параметров, а в интерфейсе он ожидается с параметрами — тип не будет реализовывать интерфейс

Реализация интерфейса

```
package main

import "fmt"

type Speaker interface { // Объявляем интерфейс
    Speak()
}

type Human struct { // Объявляем структуру
    Name string
}

func (h Human) Speak() { // Реализуем метод Speak
    fmt.Println("Привет, меня зовут", h.Name)
}

func main() {
    var s Speaker // переменная интерфейсного типа
    s = Human{"Анна"} // присваиваем значение, реализующее интерфейс

    s.Speak() // вызываем метод через интерфейс
}
```

Реализация интерфейса

Мы можем создать **несколько типов**, реализующих один и тот же интерфейс, и использовать их одинаково.

Добавим вторую структуру:

```
type Dog struct {  
    Name string  
}  
  
func (d Dog) Speak() {  
    fmt.Println("Гав! Я", d.Name)  
}
```

Реализация интерфейса

Теперь в функции `main()` мы можем сделать так:

```
var s Speaker

s = Human{"Анна"}
s.Speak()      // Привет, меня зовут Анна

s = Dog{"Шарик"}
s.Speak()      // Гав! Я Шарик
```

Это и есть **гибкость интерфейсов**: поведение определено, реализация может быть любой

Интерфейс с несколькими методам

Если интерфейс содержит несколько методов, тип должен реализовать все методы. Если хотя бы один метод не реализован — тип не будет соответствовать интерфейсу

```
type Animal interface {  
    Speak()  
    Walk()  
}  
  
type Cat struct {  
    Name string  
}  
  
func (c Cat) Speak() {  
    fmt.Println("Мяу! Я", c.Name)  
}  
  
func (c Cat) Walk() {  
    fmt.Println(c.Name, "идёт гулять")  
}
```

Интерфейсы без методов

Иногда интерфейс может быть пустым:

```
type Empty interface{}
```

Такой интерфейс принимает любое значение

Проверка реализации интерфейса

Go позволяет статически проверить соответствие типа интерфейсу с помощью компилятора, без запуска программы:

```
var _ Speaker = Human{}
```

Повторим основные шаги

- 1 Объявить интерфейс с нужными методами
- 2 Объявить структуру (или несколько структур)
- 3 Реализовать все методы интерфейса для этих структур
- 4 Использовать переменные интерфейсного типа для хранения значений, реализующих поведение
- 5 Вызывать методы через интерфейс

Мини-задача для самопроверки

Представьте, что у вас есть интерфейс `Flyer`, с методом `Fly()`:

- Создайте структуру `Bird` и `Plane`, реализующие `Fly()` по-разному
- Напишите функцию `TestFlight(f Flyer)`, вызывающую `f.Fly()`



Ваши вопросы?

Полиморфизм в языке Go



3

Полиморфизм

— это возможность работать с объектами разных типов через один и тот же интерфейс

Интерфейс, задающий поведение

Определим интерфейс:

```
type Greeter interface {  
    Greet()  
}
```

Интерфейс, задающий поведение

Создадим три разных типа, реализующих интерфейс:

```
type Person struct {
    Name string
}

func (p Person) Greet() {
    fmt.Println("Привет! Меня зовут", p.Name)
}

type Dog struct {
    Name string
}

func (d Dog) Greet() {
    fmt.Println("Гав-гав! Я", d.Name)
}

type Robot struct {
    Model string
}

func (r Robot) Greet() {
    fmt.Println("Здравствуйте. Модель", r.Model, "включена.")
}
```


Универсальная функция с интерфейсом

Напишем одну универсальную функцию, которая принимает интерфейс и вызывает у него метод:

```
func SayHello(g Greeter) {  
    g.Greet()  
}
```

Универсальная функция с интерфейсом

Теперь в `main()` можно сделать следующее:

```
func main() {  
    p := Person{Name: "Анна"}  
    d := Dog{Name: "Шарик"}  
    r := Robot{Model: "XJ-9"}  
  
    SayHello(p) // Привет! Меня зовут Анна  
    SayHello(d) // Гав-гав! Я Шарик  
    SayHello(r) // Здравствуйте. Модель XJ-9 включена.  
}
```

Мы вызываем одну и ту же функцию, но она работает с разными типами

Возможности полиморфизма

Полиморфизм позволяет:

- делать код гибким
- писать обобщённые алгоритмы
- сокращать дублирование кода
- разделять логику и реализацию

Пример полиморфизма

Создадим интерфейс, задающий поведение:

```
type Transporter interface {  
    Move(passenger string)  
}
```

Пример полиморфизма

Определим два типа:

```
type Car struct {  
    Brand string  
}  
  
func (c Car) Move(passenger string) {  
    fmt.Printf("%s едет в машине %s\n", passenger, c.Brand)  
}  
  
type Bicycle struct {  
    Color string  
}  
  
func (b Bicycle) Move(passenger string) {  
    fmt.Printf("%s едет на велосипеде цвета %s\n", passenger, b.Color)  
}
```

Пример полиморфизма

Напишем функцию, использующую интерфейс:

```
func StartRide(t Transporter, name string) {  
    t.Move(name)  
}
```

Пример полиморфизма

И вызываем:

```
func main() {  
    car := Car{Brand: "Toyota"}  
    bike := Bicycle{Color: "красный"}  
  
    StartRide(car, "Олег")  
    StartRide(bike, "Ирина")  
}  
// Вывод: Олег едет в машине Toyota, Ирина едет на велосипеде цвета красный
```

Срезы интерфейсов

Go позволяет создать срез, содержащий объекты разных типов, если все они реализуют один и тот же интерфейс:

```
func main() {  
    p := Person{Name: "Алексей"}  
    d := Dog{Name: "Бим"}  
    r := Robot{Model: "RX-78"}  
  
    greeters := []Greeter{p, d, r}  
  
    for _, g := range greeters {  
        g.Greet()  
    }  
}
```


Проверка типа во время выполнения

Конструкции **type assertion** или **type switch** позволят узнать, какой конкретно тип стоит за интерфейсной переменной:

```
func Describe(g Greeter) {  
    switch v := g.(type) {  
    case Person:  
        fmt.Println("Это человек по имени", v.Name)  
    case Dog:  
        fmt.Println("Это собака по кличке", v.Name)  
    case Robot:  
        fmt.Println("Это робот модели", v.Model)  
    default:  
        fmt.Println("Неизвестный тип")  
    }  
}
```

Что даёт полиморфизм

- 1 Возможность писать универсальные функции
- 2 Разделение логики вызова и реализации поведения
- 3 Удобную работу с множеством объектов, объединённых одним интерфейсом
- 4 Возможность добавлять новые типы, не трогая существующий код



Ваши вопросы?

Пустой интерфейс и интерфейс error



4

Пустой интерфейс

В Go пустой интерфейс — это интерфейс, не содержащий ни одного метода:

```
interface{ }
```

Переменная типа **interface{ }** может принимать значения любого типа, позволяя писать универсальный код

Пример пустого интерфейса

```
func PrintAnything(v interface{}) {  
    fmt.Println("Значение:", v)  
}
```

Теперь мы можем вызвать PrintAnything() с чем угодно:

```
func main() {  
    PrintAnything(42)  
    PrintAnything("текст")  
    PrintAnything(true)  
    PrintAnything([]int{1, 2, 3})  
}  
// Вывод: Значение: 42 Значение: текст Значение: true Значение: [1 2 3]
```

Хранение значений

Go хранит значения `interface{}` в специальной структуре, которая содержит:

1. Указатель на реальное значение
2. Информацию о его типе

Извлечение конкретного типа

При работе с `interface{}` вы теряете информацию о конкретном типе. Её можно вернуть с помощью **type assertion**:

```
func main() {  
    var x interface{} = "Привет"  
  
    s, ok := x.(string)  
    if ok {  
        fmt.Println("Строка:", s)  
    } else {  
        fmt.Println("Не строка")  
    }  
}
```


Применение пустого интерфейса

- 1 Обработка JSON
- 2 Работа с API
- 3 Реализация контейнеров
- 4 Логирование
- 5 Универсальные функции

Интерфейс error

В Go ошибки — это значения, которые реализуют интерфейс error:

```
type error interface {  
    Error() string  
}
```

Это — один из самых распространённых интерфейсов в Go

Пример интерфейса error

Рассмотрим функцию, которая делит одно число на другое:

```
func Divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("деление на ноль недопустимо")  
    }  
    return a / b, nil  
}
```

Пример интерфейса error

Теперь вызываем эту функцию:

```
func main() {  
    result, err := Divide(10, 0)  
    if err != nil {  
        fmt.Println("Ошибка:", err)  
        return  
    }  
    fmt.Println("Результат:", result)  
}  
  
// Вывод: Ошибка: деление на ноль недопустимо
```

Как создать свою ошибку

Можно создать свою структуру и реализовать метод Error():

```
type MyError struct {  
    Code int  
    Msg  string  
}  
  
func (e MyError) Error() string {  
    return fmt.Sprintf("Код ошибки %d: %s", e.Code, e.Msg)  
}
```

Как создать свою ошибку

Создаём ошибку:

```
func DoSomething() error {  
    return MyError{Code: 123, Msg: "Что-то пошло не так"}  
}
```

Как создать свою ошибку

Используем:

```
func main() {  
    err := DoSomething()  
    if err != nil {  
        fmt.Println("Ошибка:", err)  
    }  
}  
// Вывод: Ошибка: Код ошибки 123: Что-то пошло не так
```

Преимущества интерфейса error

Интерфейс error позволяет:

- передавать ошибки как значения
- обрабатывать их как обычные переменные
- оборачивать, сравнивать и логировать

Это даёт большую гибкость и упрощает контроль потока выполнения программы

Подведём итоги

- 1 `interface{}` — это универсальный тип, позволяющий хранить значения любого типа
- 2 Интерфейс `error` — это стандартный способ обработки ошибок в Go
- 3 Полезно знать, как извлекать значения из пустого интерфейса, как создавать собственные ошибки и как писать универсальные функции



Ваши вопросы?

Частые ошибки и лучшие практики

A decorative graphic in the bottom right corner consisting of two overlapping circles. The left circle is white with a dark teal number '5' inside. The right circle is dark teal and partially cut off by the edge of the frame.

5

Типичные ошибки

- 1 Интерфейс становится слишком большим
- 2 Пустой интерфейс используется неправильно или чрезмерно
- 3 Интерфейс не нужен и от него стоит отказаться

Слишком большой интерфейс

Распространённая ошибка при проектировании интерфейсов — попытка заранее создать универсальный интерфейс, содержащий все возможные варианты использования.

В результате интерфейс становится слишком обширным и теряет свою гибкость

Пример

Вместо того, чтобы создать один узкий интерфейс, мы пишем:

```
type SuperService interface {  
    Save()  
    Load()  
    Delete()  
    Export()  
    Import()  
    Validate()  
    Print()  
}
```

Слишком большой интерфейс

Если интерфейс слишком большой, любая реализация должна либо тянуть за собой ненужные методы, либо придумывать фиктивные заглушки. Это нарушает принцип **Single Responsibility**

Как исправить

Лучше разбить интерфейс на маленькие, специализированные интерфейсы:

```
type Saver interface {  
    Save()  
}  
  
type Loader interface {  
    Load()  
}  
  
type Validator interface {  
    Validate()  
}
```

Теперь реализация может выбирать, какие именно интерфейсы ей нужны

Неправильное использование пустого интерфейса

В большинстве случаев `interface{}` используют не по назначению:

```
type User struct {  
    ID    interface{}  
    Name  interface{}  
    Age   interface{}  
}
```

Неправильное использование пустого интерфейса

Использование `interface{}` оправдано только если тип неизвестен заранее.

Например, при парсинге JSON или при логировании произвольных данных.

В остальных случаях нужно выбирать конкретный тип

Как исправить

```
type User struct {  
    ID    int  
    Name  string  
    Age   int  
}
```

Теперь мы точно знаем, какие значения у нас есть

Где пустой интерфейс допустим

- В функциях типа `fmt.Println()`
- При работе с неструктурированными входными данными
- При реализации обобщённых коллекций до появления generics

Интерфейс не нужен

Напишем простую библиотеку, в которой есть только одна реализация какого-либо поведения:

```
type FileService struct {}

func (fs FileService) Save(path string, data []byte) error {
    // сохранение в файл
    return nil
}
```

Интерфейс не нужен

Распространённый вариант написания:

```
type Saver interface {  
    Save(path string, data []byte) error  
}
```

Интерфейс не нужен

Далее создаётся:

```
type FileService struct {}  
  
func (fs FileService) Save(...) error { ... }
```

И используется интерфейс, даже если нет ни одной альтернативной реализации. Это мешает чтению кода и делает архитектуру более хрупкой

Пример оправданного интерфейса

Интерфейс пригодится, если вы хотите подменить реализацию. Самый частый случай — тестирование. Пример:

```
type Storage interface {  
    Save(path string, data []byte) error  
}
```

Теперь можно внедрить в программу как `FileStorage`, так и `MockStorage`, и в тестах не трогать файловую систему

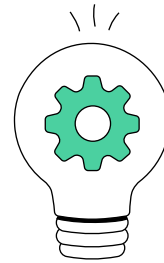
Подведём итоги

- 1 Не создавайте интерфейсы на будущее
- 2 Не пишите большие интерфейсы, объединяющие всё подряд
- 3 Не злоупотребляйте `interface{}`
- 4 Go не требует интерфейсов ради интерфейсов



Ваши вопросы?

Практика



Цели

Выполнение задач поможет вам лучше понять:

- Как задавать интерфейсы для описания поведения
- Как реализовывать их в разных типах
- Как использовать полиморфизм для написания универсальных функций, не зависящих от конкретных типов

Задача 1

Написать программу на Go, в которой:

1. Определён интерфейс Shape с методом Area() float64
2. Созданы две структуры: Rectangle (прямоугольник) и Circle (круг)
 - Rectangle хранит ширину и высоту (Width, Height)
 - Circle хранит радиус (Radius)
3. Для обоих типов реализован метод Area():
 - У Rectangle — $\text{Width} * \text{Height}$
 - У Circle — $\pi * \text{Radius}^2$ (можно использовать math.Pi)
4. Написана функция TotalArea(shapes []Shape) float64, принимающая срез Shape и возвращающая сумму площадей всех фигур
5. В main() создаётся несколько прямоугольников и кругов, суммарная площадь выводится на экран

```

package main

import (
    "fmt"
    "math"
)

type Shape interface { // 1. Объявляем интерфейс Shape
    Area() float64 // Метод, возвращающий площадь фигуры
}

type Rectangle struct { // 2. Объявляем структуру Rectangle
    Width, Height float64
}

type Circle struct { // 2. Объявляем структуру Circle
    Radius float64
}

func (r Rectangle) Area() float64 { // 3. Реализация метода Area для Rectangle
    return r.Width * r.Height // Площадь прямоугольника = ширина * высота
}

func (c Circle) Area() float64 { // 3. Реализация метода Area для Circle
    // Площадь круга =  $\pi * r^2$ 
    return math.Pi * c.Radius * c.Radius
}

func TotalArea(shapes []Shape) float64 { // 4. Функция суммирует площади всех фигур из среза
    var total float64
    for _, s := range shapes {
        total += s.Area() // Для каждого элемента вызываем метод Area()
    }
    return total
}

func main() {
    rect1 := Rectangle{Width: 3, Height: 4} // Создаём несколько фигур
    rect2 := Rectangle{Width: 5, Height: 2}
    circle1 := Circle{Radius: 2.5}

    shapes := []Shape{rect1, rect2, circle1} // Собираем их в срез интерфейсного типа

    fmt.Printf("Суммарная площадь: %.2f\n", TotalArea(shapes)) // Вычисляем и выводим суммарную площадь
}

```

Задача 2

Написать программу на Go, в которой:

1. Определён интерфейс `Notifier` с методом `Notify(message string)`
2. Созданы две структуры: `EmailNotifier` и `SMSNotifier`
 - `EmailNotifier` хранит поле `EmailAddress string`
 - `SMSNotifier` хранит поле `PhoneNumber string`
3. Для обоих типов реализован метод `Notify(message string)`, который выводит в консоль текст:
 - Email to <EmailAddress>: <message>
 - SMS to <PhoneNumber>: <message>
4. Есть функция `SendNotifications(nots []Notifier, msg string)`, которая перебирает срез `Notifier` и вызывает `Notify(msg)` у каждого элемента
5. В `main()` созданы экземпляры `EmailNotifier` и `SMSNotifier` и им разослано тестовое сообщение

```
package main

import "fmt"

type Notifier interface { // 1. Объявляем интерфейс Notifier
    Notify(message string) // Метод отправки уведомления
}

type EmailNotifier struct { // 2. Объявляем EmailNotifier
    EmailAddress string
}

type SMSNotifier struct { // 2. Объявляем SMSNotifier
    PhoneNumber string
}

func (e EmailNotifier) Notify(message string) { // 3. Реализация Notify для EmailNotifier
    fmt.Printf("Email to %s: %s\n", e.EmailAddress, message) // В реальном приложении здесь был бы SMTP-сервер
}

func (s SMSNotifier) Notify(message string) { // 3. Реализация Notify для SMSNotifier
    fmt.Printf("SMS to %s: %s\n", s.PhoneNumber, message) // Здесь мог бы быть вызов SMS-шлюза
}

func SendNotifications(nots []Notifier, msg string) { // 4. Универсальная функция для рассылки уведомлений
    for _, n := range nots {
        n.Notify(msg) // Для каждого Notifier вызываем Notify
    }
}

func main() {
    email := EmailNotifier{EmailAddress: "user@example.com"} // 5. Создаём конкретные Notifier
    sms := SMSNotifier{PhoneNumber: "+71234567890"}

    notifiers := []Notifier{email, sms} // Собираем их в срез интерфейсного типа

    message := "Ваш отчёт готов!" // Тестовое сообщение

    SendNotifications(notifiers, message) // Отправляем всем сразу
}
```


Итоги занятия

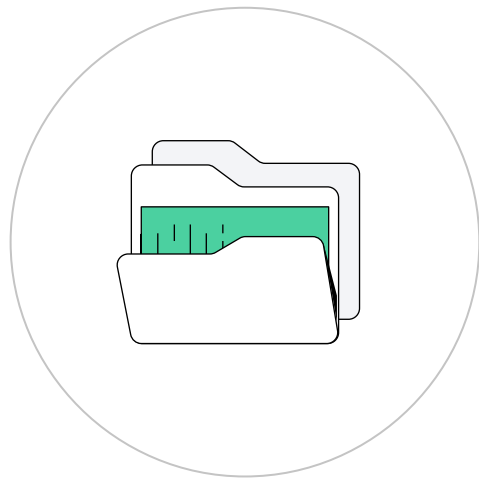
Сегодня мы:

- Узнали, как объявлять интерфейсы в языке Go
- Поняли, как интерфейсы позволяют реализовывать поведение без указания конкретных типов
- Выяснили, как работает реализация интерфейса через методы структур
- Познакомились с полиморфизмом в Go
- Разобрались, для чего используется пустой интерфейс
- Изучили, как устроен и применяется стандартный интерфейс `error`
- Рассмотрели частые ошибки при использовании интерфейсов и поняли, как их избежать



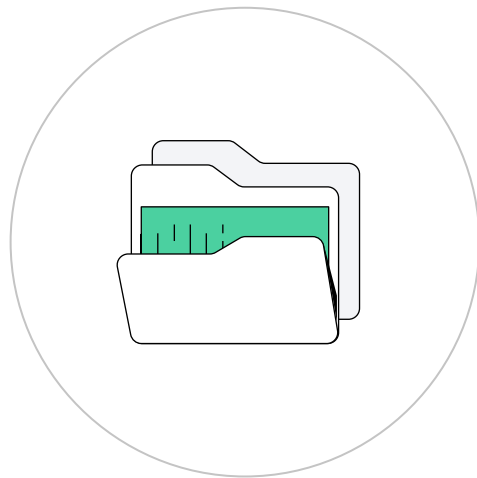
Рекомендации

- Практикуйтесь: пробуйте выделить интерфейс, даже если он кажется лишним
- Поработайте с коллекциями, хранящими элементы разных типов, и попробуйте обработать их через `interface{}`
- Посмотрите, как интерфейсы применяются в стандартной библиотеке Go, особенно в модулях `fmt`, `sort`, `net/http`



Дополнительные материалы

- [Официальная документация](#) по интерфейсам
- [Разбор интерфейсов](#) с примерами
- [Интерактивная песочница](#) для экспериментов в браузере





Ваши вопросы?

Интерфейсы в Go

Матвей Ефимов
Эксперт в Go, Python, SQL

